Deep Q Network (DQN)

Kianté Brantley
CS 1840 Fall 2026

Announcements

HW3 is out and due 10/29!

Midterm will be on Mon 10/26!

Check the course website for Office Hours:
https://harvard-cs-1840.github.io/2026/

Recap: Bellman operator

Value iteration

$$Q^{t+1} \Leftarrow \mathcal{T} Q^{t}$$

table

Recap: Q-learning

Tabular Q Learning: maintain a table $\hat{Q}$ of size S x A

$$\hat{Q}(s,a) \Leftarrow \hat{Q}(s,a) + \eta \left(\underbrace{r + \gamma \max_{a'} \hat{Q}(s',a')}_{\text{target}} - \underbrace{\hat{Q}(s,a)}_{\text{prediction}} \right)$$

Data collection via ϵ-greedy:

Recap: Q-learning

Tabular Q Learning: maintain a table $\hat{Q}$ of size S x A

$$\hat{Q}(s,a) \Leftarrow \hat{Q}(s,a) + \eta \left(r + \gamma \max_{a'} \hat{Q}(s',a') - \hat{Q}(s,a) \right)$$

Data collection via ϵ-greedy:

W/ prob ϵ, select action uniform randomly

W/ prob $1-\epsilon$, select greedy action $\arg\max_a \hat{Q}(s,a)$

Today

Consider large-scale MDPs,

how to estimate $Q^\star(s, a)$ using function approximation (e.g., neural network)

Deep Q-network (DQN) is the earliest example of showing Deep Learning + RL is powerful

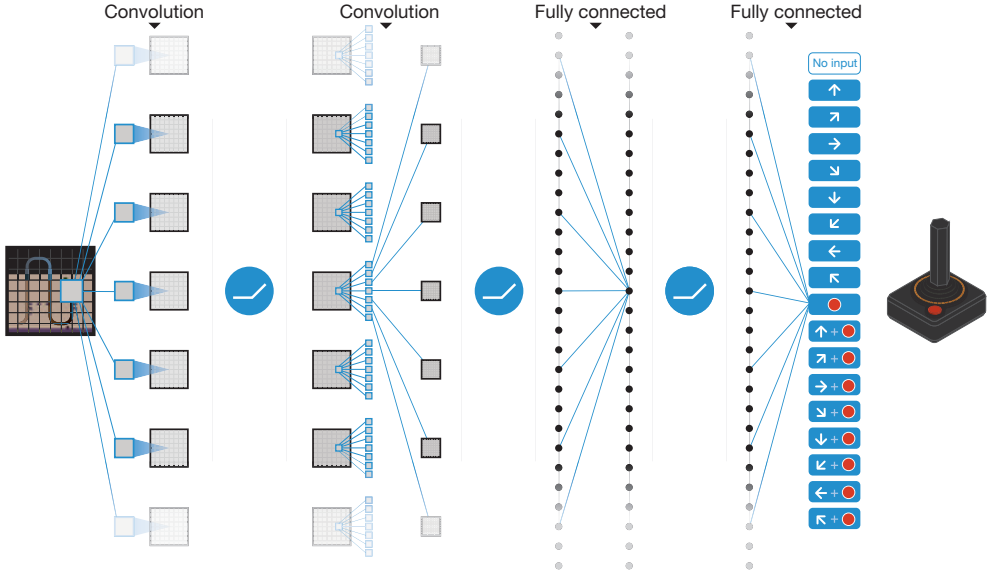
doi:10.1038/nature14236

Human-level control through deep reinforcement learning

Volodymyr Mnih[1]*, Koray Kavukcuoglu[1]*, David Silver[1]*, Andrei A. Rusu[1], Joel Veness[1], Marc G. Bellemare[1], Alex Graves[1], Martin Riedmiller[1], Andreas K. Fidjeland[1], Georg Ostrovski[1], Stig Petersen[1], Charles Beattie[1], Amir Sadik[1], Ioannis Antonoglou[1], Helen King[1], Dharshan Kumaran[1], Daan Wierstra[1], Shane Legg[1] & Demis Hassabis[1]

Deep Q-network (DQN)

Deep Learning + RL = Deep Reinforcement Learning



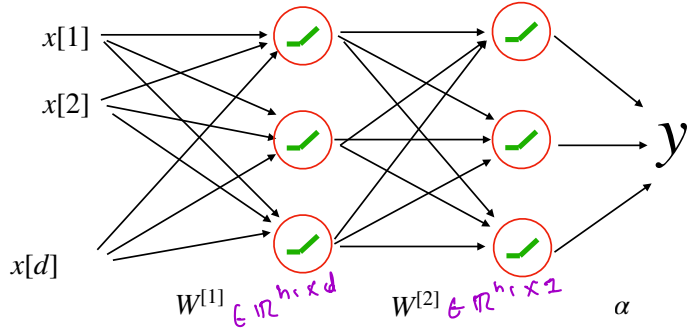
Outline:

1. Q Learning w/ function approximation

2. Replay buffer, batch optimization and target network

Q-Learning w/ function approximation

We will model $Q^\star$ using a function approximator



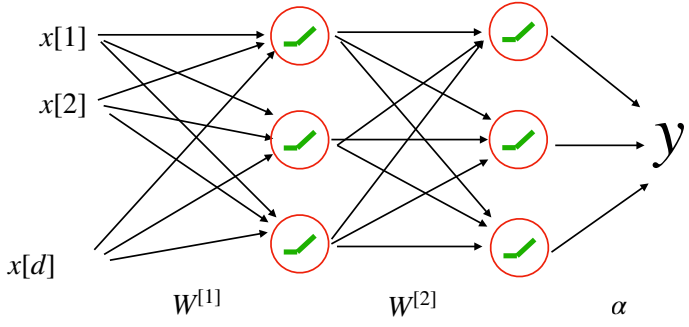
$$x = [s^\top, a]^\top \in \mathbb{R}^d$$

$$y = \alpha^\top \text{ReLU}\left(W^{[2]}\text{ReLU}\left(W^{[1]}x\right)\right) + b$$

$$Q_\theta(s, a) : S \times A \to \mathbb{R}$$

Q-Learning w/ function approximation

We will model $Q^\star$ using a function approximator



$$Q_\theta(s,a) : S \times A \to \mathbb{R}$$

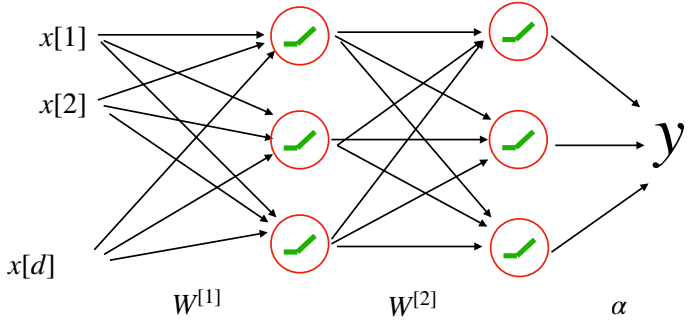
Assumption: differentiable $\nabla_\theta Q_\theta(s,a)$

$$x = [s^\top, a]^\top$$

$$y = \alpha^\top \text{ReLU}\left(W^{[2]}\text{ReLU}\left(W^{[1]}x\right)\right) + b$$

Q-Learning w/ function approximation

We will model $Q^\star$ using a function approximator



$$x = [s^\top, a]^\top$$

$$y = \alpha^\top \text{ReLU}\left(W^{[2]}\text{ReLU}\left(W^{[1]}x\right)\right) + b$$

$$Q_\theta(s, a) : S \times A \to \mathbb{R}$$

Assumption: differentiable $\nabla_\theta Q_\theta(s, a)$

(The DQN paper uses ConvNet as s is an image frame of the game)



Attempt 1: Q Learning w/ function approximation

Initialize θ^0. Set initial state $s \in \mathcal{S}$

For t = 0 to T

Attempt 1: Q Learning w/ function approximation

Initialize θ^0. Set initial state $s \in \mathcal{S}$

For t = 0 to T

$[s, a, r, i) = \text{env.step}(a)$

Take action a based on ϵ-greedy of Q_{θ^t}, get reward r and next state $s' \sim P(\cdot \mid s, a)$

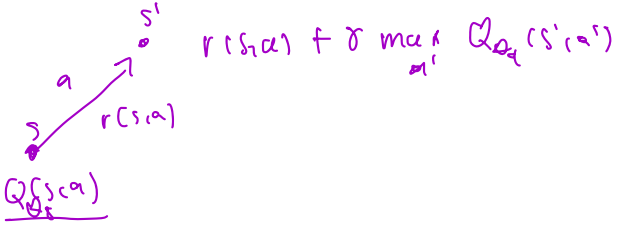
Attempt 1: Q Learning w/ function approximation

Initialize θ^0. Set initial state $s \in \mathcal{S}$

For t = 0 to T

Take action a based on ϵ-greedy of Q_{θ^t}, get reward r and next state $s' \sim P(\cdot \mid s, a)$

Form Q-target $r + \gamma \max_{a'} Q_{\theta_t}(s', a')$



Attempt 1: Q Learning w/ function approximation

Initialize θ^0. Set initial state $s \in \mathcal{S}$

For t = 0 to T

Take action a based on ϵ-greedy of Q_{θ^t}, get reward r and next state $s' \sim P(\cdot \mid s, a)$

Form Q-target $r + \gamma \max_{a'} Q_{\theta_t}(s', a')$

Update to θ^{t+1}: ← SGD on MSE loss

Attempt 1: Q Learning w/ function approximation

Initialize θ^0. Set initial state $s \in \mathcal{S}$

For t = 0 to T

Take action a based on ϵ-greedy of Q_{θ^t}, get reward r and next state $s' \sim P(\cdot \mid s, a)$

Form Q-target $r + \gamma \max_{a'} Q_{\theta_t}(s', a')$

Update to θ^{t+1}:

Set $s \Leftarrow s'$

Q Learning w/ function approximation

Update parameters using SGD on the Bellman error loss

Q Learning w/ function approximation

Update parameters using SGD on the Bellman error loss

$$\ell_{be}(\theta) := \big(Q_\theta(s,a) - y\big)^2, \text{ where } y = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} \max_{a'} Q_\theta(s',a')$$

prediction target

$$\big[Q_\theta(s,a) - (\mathcal{T}Q_\theta)(s,a)\big]^2$$

$$\nabla \ell_{BE}(\theta) = 2\big(Q_\theta(s,a) - y\big) \circ \nabla_\theta Q_\theta(s,a)$$

$$\widetilde{\nabla} \ell_{BE}(\theta) = 2\Big[Q_\theta(s,a) - \big[r(s,a) + \gamma \max_{a'} Q_\theta[s',a']\big]\Big]$$

$$\theta = \theta - \eta \widetilde{\nabla} \ell_{BE}(\theta)$$

Issues of this simple approach

1. Inefficient — it throws away all historical data
(your network could forget old experiences, i.e., catastrophic forgetting)

Issues of this simple approach

1. Inefficient — it throws away all historical data
(your network could forget old experiences, i.e., catastrophic forgetting)

2. instablity — Training is quite unstable (we saw it from the past Cartpole Demo)

Outline:

1. Q Learning w/ function approximation

2. Replay buffer, batch optimization and target network

Q-Learning w/ function approximation

First improvement: Replay buffer

$$\mathscr{D}_{rb} = \begin{bmatrix} \cdots \\ (s, a, r, s') \\ \cdots \\ \cdots \end{bmatrix}$$

for $t = 0$ to T
- Take a greedy action
- form a target
- update θ_t
- update $s \leftarrow s'$

(s, a, r, s')

A dataset that contains all historical
State-action-reward-next state tuples

Q-Learning w/ function approximation

With replay buffer, we can use mini-batch SGD to update Bellman error loss

Q-Learning w/ function approximation

With replay buffer, we can use mini-batch SGD to update Bellman error loss

Given Q_{θ^t} and replay buffer $\mathcal{D}_{rb}$, randomly sample a mini-batch $\mathcal{B}$ from $\mathcal{D}_{rb}$

$$\mathcal{D}_{rb} = \{(s_i, a_i, r_i, s'_i)\}_{i=1}^{N}$$

$$\frac{1}{|\mathcal{B}|} \sum_{(s,a,r,s') \in \mathcal{B}} \left[(Q_\theta(s,a) - r - \max_{a'} Q_{\theta^t}(s', a') \right]^2$$

Q-Learning w/ function approximation

With replay buffer, we can use mini-batch SGD to update Bellman error loss

Given Q_{θ^t} and replay buffer $\mathscr{D}_{rb}$, randomly sample a mini-batch $\mathscr{B}$ from $\mathscr{D}_{rb}$

$$\theta^{t+1} = \theta^t - \eta \frac{1}{|\mathscr{B}|} \sum_{(s,a,r,s') \in \mathscr{B}} \left(Q_{\theta^t}(s,a) - r - \max_{a'} Q_{\theta^t}(s',a') \right) \nabla_\theta Q_{\theta_t}(s,a)$$

target

prediction

gradient

learning rate

sample from buffer

Q-Learning w/ function approximation

Second improvement: Target network (making Q learning more stable)

Q-Learning w/ function approximation

Second improvement: Target network (making Q learning more stable)

Recall that Q learning can be understood as running SGD on an **evolving** loss function

Q-Learning w/ function approximation

Second improvement: Target network (making Q learning more stable)

Recall that Q learning can be understood as running SGD on an **evolving** loss function

$$\ell_{be}(\theta) := \big(Q_\theta(s,a) - y\big)^2, \text{ where } y = \underbrace{r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} \max_{a'} Q_\theta(s',a')}_{\text{regression target}}$$

Q_θ: prediction; y: target; Q_θ: t

Q-Learning w/ function approximation

Second improvement: Target network (making Q learning more stable)

Recall that Q learning can be understood as running SGD on an **evolving** loss function

$$\ell_{be}(\theta) := \big(Q_\theta(s,a) - y\big)^2, \text{ where } y = \underbrace{r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} \max_{a'} Q_\theta(s',a')}_{\text{regression target}}$$

Normal supervised learning

$\ell(\theta) = (f(x) - y)^2$ x = input y = target

Source of instability: target changes immediately whenever we update θ

Q-Learning w/ function approximation

Second improvement: Target network (making Q learning more stable)

Introducing target network $Q_{\tilde{\theta}}$ to **slow down** the evolution of the BE loss

(e.g., set $\tilde{\theta}$ as a copy of an older version of θ)

Q-Learning w/ function approximation

Second improvement: Target network (making Q learning more stable)

Introducing target network $Q_{\tilde{\theta}}$ to **slow down** the evolution of the BE loss

(e.g., set $\tilde{\theta}$ as a copy of an older version of θ)

$$\ell_{be}(\theta) := \left(Q_\theta(s,a) - y\right)^2, \text{ where } y = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} \max_{a'} Q_{\tilde{\theta}}(s', a')$$

Q-Learning w/ function approximation

Second improvement: Target network (making Q learning more stable)

Introducing target network $Q_{\tilde{\theta}}$ to **slow down** the evolution of the BE loss

(e.g., set $\tilde{\theta}$ as a copy of an older version of θ)

$$\ell_{be}(\theta) := \left(Q_\theta(s,a) - y\right)^2, \text{ where } y = r(s,a) + \gamma \mathbb{E}_{s' \sim P(\cdot|s,a)} \max_{a'} Q_{\tilde{\theta}}(s',a')$$

Use a target network that slowly catches up θ

Attempt 2: Deep Q network (DQN)

Initialize θ and replay buffer $\mathcal{D}_{rb}$. Set $\tilde{\theta} = \theta$, Set initial state $s \in \mathcal{S}$

While true:

Take action a based on ϵ-greedy of Q_θ, get reward r and next state $s' \sim P(\cdot \mid s, a)$

Attempt 2: Deep Q network (DQN)

Initialize θ and replay buffer $\mathcal{D}_{rb}$. Set $\tilde{\theta} = \theta$, Set initial state $s \in \mathcal{S}$

While true:

Take action a based on ϵ-greedy of Q_θ, get reward r and next state $s' \sim P(\cdot \mid s, a)$

Add (s, a, r, s') to $\mathcal{D}_{rb}$

Attempt 2: Deep Q network (DQN)

Initialize θ and replay buffer $\mathscr{D}_{rb}$. Set $\tilde{\theta} = \theta$, Set initial state $s \in \mathcal{S}$

While true:

Take action a based on ϵ-greedy of Q_θ, get reward r and next state $s' \sim P(\cdot \mid s, a)$

Add (s, a, r, s') to $\mathscr{D}_{rb}$

Sample mini-batch $\mathscr{B}$ from $\mathscr{D}_{rb}$

Attempt 2: Deep Q network (DQN)

Initialize θ and replay buffer $\mathscr{D}_{rb}$. Set $\tilde{\theta} = \theta$, Set initial state $s \in \mathcal{S}$

While true:

Take action a based on ϵ-greedy of Q_θ, get reward r and next state $s' \sim P(\cdot \mid s, a)$

Add (s, a, r, s') to $\mathscr{D}_{rb}$

Sample mini-batch $\mathscr{B}$ from $\mathscr{D}_{rb}$

Update parameters:

$$\theta \Leftarrow \theta - \eta \frac{1}{|\mathscr{B}|} \sum_{(s,a,r,s') \in \mathscr{B}} \left(Q_\theta(s,a) - r - \max_{a'} Q_{\tilde{\theta}}(s', a') \right) \nabla_\theta Q_\theta(s,a)$$

Attempt 2: Deep Q network (DQN)

Initialize θ and replay buffer $\mathcal{D}_{rb}$. Set $\tilde{\theta} = \theta$, Set initial state $s \in \mathcal{S}$

While true:

Take action a based on ϵ-greedy of Q_θ, get reward r and next state $s' \sim P(\cdot \mid s, a)$

Add (s, a, r, s') to $\mathcal{D}_{rb}$

Sample mini-batch $\mathcal{B}$ from $\mathcal{D}_{rb}$

Update parameters:

$$\theta \Leftarrow \theta - \eta \frac{1}{|\mathcal{B}|} \sum_{(s,a,r,s') \in \mathcal{B}} \left(Q_\theta(s,a) - r - \max_{a'} Q_{\tilde{\theta}}(s', a') \right) \nabla_\theta Q_\theta(s,a)$$

Every C step, set $\tilde{\theta} = \theta$ (option: $\tilde{\theta} = (1-\alpha)\theta + \alpha\tilde{\theta}$)

When C is large…

DQN is performing SGD for standard regression between two target network updates..

$$\min_{\theta} \sum_{s,a,r,s' \in \mathcal{D}_{rb}} \left(Q_\theta(s,a) - \left(r + \max_{a'} Q_{\tilde{\theta}}(s',a') \right) \right)^2$$

When C is large…

DQN is performing SGD for standard regression between two target network updates..

$$\min_{\theta} \sum_{s,a,r,s' \in \mathcal{D}_{rb}} \left(Q_\theta(s,a) - \left(r + \max_{a'} Q_{\tilde{\theta}}(s',a') \right) \right)^2$$

Q: What is the Bayes optimal of this regression problem?

$Y = r + \gamma \max_{a'} Q_{\tilde{\theta}}(s',a')$

$Q_{\theta^*}(s,a) = \mathbb{E}[Y \mid s,a]$

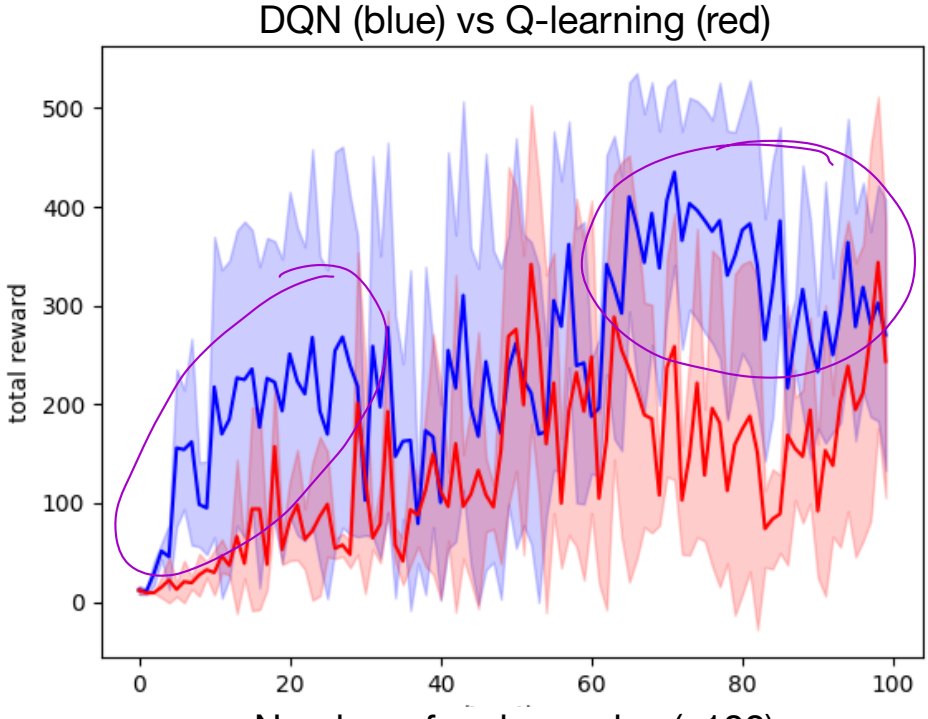
$= r(s,a) + \gamma \mathbb{E}_{s' \sim P}\left[\max_{a'} Q_{\tilde{\theta}}(s',a') \right]$

$= (\mathcal{T} Q_{\tilde{\theta}})(s,a)$

Tabular

$Q^{t+1} = \mathcal{T} Q^t$

DQN vs Naive Q-learning



BLUE: DQN

RED: Q-learning

Summary

1. Using function approximation to handle large state space

2. Making Q-learning closer to the supervised learning (i.e., regression) framework:

- Replay buffer + mini-batch SGD
- Target network to simulate a standard regression setting

Feedback and Attendance

https://bit.ly/4gYn2nC

